

PLANO DE TESTES

PIX LEGDER

ENGENHARIA DE SISTEMAS DISTRIBUÍDOS

ALUNOS:

- **HENRIQUE BANDEIRA**
- **MATHEUS YAGO LIMA DE FREITAS**
- **JOÃO VICTOR DA SILVA CIRILO**
- **JOÃO VICTOR OLIVEIRA DE LIMA**
- **LUIZ HENRIQUE DOS SANTOS SOUZA**

30/03/2026
João Pessoa

Sumário

Introdução.....	3
Escopo.....	4
Escopo do Teste.....	4
Fora do Escopo.....	4
Casos de Teste.....	5
Critérios de Aceitação.....	6
Ambiente de Testes.....	7
5.1 Configuração do Ambiente.....	7
5.2 Ferramentas Utilizadas.....	7
5.3 Dados de Teste.....	7

Introdução

O sistema **PIX Ledger** consiste em uma aplicação responsável pelo gerenciamento de usuários e registro de transações financeiras, simulando operações similares ao ecossistema de pagamentos instantâneos. O sistema permite o cadastro e consulta de usuários, bem como a realização e consulta de transações, garantindo o controle e a rastreabilidade das operações realizadas.

Dado o caráter financeiro da aplicação, é essencial assegurar a integridade, consistência e confiabilidade dos dados manipulados, especialmente nas operações de transferência de valores. Nesse contexto, o presente plano de testes tem como objetivo definir estratégias, critérios e casos de teste que possibilitem validar o correto funcionamento do sistema, identificando possíveis falhas e garantindo que os requisitos funcionais sejam atendidos.

Este documento abrange os testes das principais funcionalidades do sistema, incluindo operações de CRUD de usuários e processamento de transações, contemplando diferentes cenários de uso, entradas válidas e inválidas, e verificações de regras de negócio.

Objetivo do Teste

O objetivo deste plano de testes é verificar se o sistema **PIX Ledger** atende corretamente aos requisitos funcionais especificados, garantindo que as operações de cadastro e consulta de usuários, bem como a criação e consulta de transações, sejam executadas de forma consistente, segura e sem falhas.

Busca-se validar o comportamento do sistema diante de diferentes cenários, incluindo entradas válidas e inválidas, assegurando o correto tratamento de erros e a integridade dos dados persistentes. Além disso, os testes visam confirmar que as regras de negócio associadas às transações financeiras — como validação de usuários e consistência das operações — estão sendo devidamente aplicadas.

Escopo

O presente plano de testes contempla a validação das principais funcionalidades do sistema **PIX Ledger**, com foco na verificação do correto funcionamento das operações relacionadas ao gerenciamento de usuários e transações financeiras.

Escopo do Teste

Estão incluídos neste plano os seguintes itens:

- Cadastro de usuários, incluindo validação de dados de entrada
- Consulta de usuários por identificador
- Criação de novas transações financeiras
- Consulta de transações por ID
- Validação das regras de negócio associadas às transações
- Verificação da persistência correta dos dados no banco de dados
- Tratamento de erros para entradas inválidas ou inconsistentes
- Testes de carga sobre os serviços principais da aplicação
- Testes de resiliência com falha induzida de contêineres
- Observação de métricas e comportamento do sistema em ambiente instrumentado

Fora do Escopo

Não fazem parte deste plano de testes:

- Integrações com sistemas externos ou APIs de terceiros
- Testes de interface gráfica (caso aplicável)
- Aspectos de segurança avançada (como criptografia e autenticação robusta)
- Estratégias de backup e recuperação de dados

Casos de Teste

ID	Funcionalidade	Cenário	Entrada	Saída Esperada
CT01	Cadastro de usuário	Cadastro com sucesso	Dados válidos	Usuário criado com sucesso
CT02	Cadastro de usuário	Usuário duplicado	Mesmo identificador já existente	Erro de duplicidade
CT03	Cadastro de usuário	Campos obrigatórios vazios	Dados incompletos	Erro de validação
CT04	Cadastro de usuário	Formato inválido	Dados fora do padrão esperado	Erro de validação
CT05	Cadastro de usuário	Limite de tamanho excedido	Campos com tamanho acima do permitido	Erro de validação
CT06	Transação	Transação realizada com sucesso	Dados válidos e saldo suficiente	Transação registrada
CT07	Transação	Saldo insuficiente	Valor maior que saldo disponível	Erro de saldo insuficiente
CT08	Transação	Usuário inexistente	ID de usuário inválido	Erro de usuário não encontrado

ID	Funcionalidade	Cenário	Entrada	Saída Esperada
CT09	Consulta transação	Consulta com sucesso	ID válido	Retorna dados da transação
CT10	Consulta transação	Transação inexistente	ID inválido	Retorno vazio ou erro
CT11	Desempenho	Execução sob carga	Múltiplas requisições concorrentes	Sistema responde com latência aceitável
CT12	Resiliência	Falha controlada de contêiner	Indisponibilidade temporária de serviço	Sistema degrada de forma controlada e se recupera

Critérios de Aceitação

Para que o sistema **PIX Ledger** seja considerado aprovado, os seguintes critérios devem ser atendidos:

- Todas as funcionalidades principais (cadastro e consulta de usuários, criação e consulta de transações) devem ser executadas sem erros críticos
- Os dados devem ser corretamente persistidos no banco de dados após cada operação
- O sistema deve retornar respostas adequadas para entradas inválidas, incluindo mensagens de erro claras
- As regras de negócio devem ser respeitadas, especialmente em operações de transação
- As consultas devem retornar informações corretas e consistentes com os dados armazenados
- Nenhum comportamento inesperado (falhas, inconsistências ou travamentos) deve ocorrer durante os testes

Ambiente de Testes

Os testes do sistema **PIX Ledger** serão realizados em um ambiente controlado, garantindo condições adequadas para validação das funcionalidades.

5.1 Configuração do Ambiente

- Execução da aplicação em ambiente local
- Banco de dados configurado e acessível
- API disponível para requisições

5.2 Ferramentas Utilizadas

- Ferramentas de requisição HTTP (Insomnia REST)
- Sistema de gerenciamento de banco de dados (PostgreSQL)
- Ambiente de desenvolvimento utilizado no projeto
- k6 para testes de carga e resiliência
- Scripts shell (`run-tests.sh` e `run-resilience-full.sh`) para automação da execução dos testes
- SigNoz para observação de métricas e comportamento dos serviços

5.3 Dados de Teste

- Dados fictícios para cadastro de usuários
- Diferentes cenários de saldo para simulação de transações
- Identificadores válidos e inválidos para testes de consulta

Testes Não Funcionais

Com o objetivo de complementar os testes funcionais previamente definidos, também foram realizados testes não funcionais voltados à avaliação de desempenho sob carga e à análise de resiliência da aplicação diante de falhas controladas em seus contêineres.

Esses testes foram executados em ambiente local containerizado, utilizando scripts automatizados e geração de métricas de execução, permitindo observar o comportamento do sistema em condições de estresse e indisponibilidade parcial de componentes.

6.1 Teste de Carga

O teste de carga teve como finalidade submeter o sistema a um volume crescente de requisições simultâneas, avaliando tempo de resposta, taxa de falhas, quantidade de requisições processadas, número de usuários virtuais suportados e consumo de rede.

Para execução do teste, foi utilizado o script `run-tests.sh`, executado no terminal Linux ou no Git Bash no Windows. Antes da execução, foi necessário conceder permissão ao script com o comando:

```
chmod +x run-tests.sh
```

Em seguida, o teste foi iniciado com:

```
./run-tests.sh
```

6.1.1 Resultados dos Testes de Carga

Os principais resultados observados foram os seguintes:

- Tempo médio de resposta HTTP (`http_req_duration`): **29,08 ms**
- Mediana de resposta: **6,98 ms**
- Percentil 90: **44,07 ms**
- Percentil 95: **85,32 ms**
- Tempo máximo observado: **13,33 s**
- Taxa de falha HTTP (`http_req_failed`): **42,11%**
- Total de requisições HTTP: **36.060**
- Vazão aproximada: **94,71 requisições por segundo**
- Total de iterações concluídas: **9.965**
- Duração média por iteração: **1,62 s**
- Número máximo de usuários virtuais simultâneos: **70**
- Dados recebidos: **14 MB**
- Dados enviados: **5,3 MB**

O teste foi concluído em aproximadamente **6 minutos e 20 segundos**, com **9.965 iterações completas** e sem interrupções.

6.1.2 Análise dos Testes de Carga

Os resultados indicam que o sistema apresentou tempos de resposta satisfatórios na maior parte das requisições, com percentil 95 inferior a 100 ms, o que demonstra boa responsividade média sob carga.

Entretanto, a taxa de falha HTTP de **42,11%** é elevada e sugere a ocorrência de erros funcionais ou de integração durante a execução, exigindo investigação adicional. Esse resultado impede que o teste de carga seja considerado plenamente satisfatório, embora ele tenha sido útil para identificar gargalos e comportamentos instáveis sob pressão.

Assim, conclui-se que o sistema suportou volume significativo de requisições, mas ainda necessita de ajustes para redução da taxa de falhas em cenários de carga mais intensa.

6.2 Teste de Resiliência

O teste de resiliência teve como finalidade verificar o comportamento do sistema quando submetido a falhas controladas na infraestrutura, especialmente por meio da interrupção temporária de contêineres essenciais da aplicação.

Para execução do teste, foi utilizado o script `run-resilience-full.sh`, também executado no terminal Linux ou Git Bash no Windows. Antes da execução, foi necessário conceder permissão ao arquivo com:

```
chmod +x run-resilience-full.sh
```

Em seguida, o teste foi iniciado com:

```
./run-resilience-full.sh
```

Durante a execução, foram simuladas falhas em contêineres do ambiente, permitindo observar a capacidade do sistema de continuar operando, degradar parcialmente ou se recuperar após o restabelecimento dos serviços.

6.2.1 Resultados dos Testes de Resiliência

Os resultados consolidados foram:

- Percentil 95 do tempo de resposta HTTP: **190,75 ms**
- Taxa de falha HTTP (`http_req_failed`): **32,42%**
- Total de requisições HTTP: **47.909**
- Vazão aproximada: **133,06 requisições por segundo**
- Total de verificações (`checks_total`): **47.909**
- Verificações bem-sucedidas: **83,27%**

- Verificações com falha: **16,72%**
- Tempo médio de resposta HTTP: **59,12 ms**
- Duração média por iteração: **1,46 s**
- Máximo de usuários virtuais simultâneos: **50**
- Dados recebidos: **20 MB**
- Dados enviados: **7,8 MB**

Resultados específicos por operação:

- `POST /usuarios -> 200 ou 201: 84% de sucesso`
- `POST /transacoes -> 200 ou 201: 99% de sucesso`
- `GET /usuarios/{id}/transacoes -> 200: 99% de sucesso`
- `GET /transacoes/{id} -> 200: 99% de sucesso`
- `GET ledger balance`: comportamento aceitável dentro da política definida para falhas
- `GET ledger statement`: comportamento aceitável dentro da política definida para falhas

6.2.2 Análise dos Testes de Resiliência

O teste de resiliência apresentou resultado melhor que o teste de carga no critério de tempo de resposta, com percentil 95 de **190,75 ms**, dentro do limite esperado para o cenário configurado. A taxa de falha HTTP de **32,42%**, embora ainda relevante, permaneceu dentro do limiar definido para o teste.

As operações de criação de transações e consultas principais mantiveram alto índice de sucesso, o que sugere que o sistema preservou parte significativa de sua funcionalidade mesmo diante de falhas induzidas.

De modo geral, o sistema demonstrou capacidade parcial de suportar falhas controladas, mantendo a execução de diversas operações e apresentando degradação localizada em determinados endpoints.

6.3 Teste de Integração Observável

Com o objetivo de validar a integração entre os serviços distribuídos da aplicação, foi realizada uma análise baseada em observabilidade utilizando a ferramenta SigNoz.

Diferentemente dos testes tradicionais de integração, essa abordagem permite verificar o comportamento real do sistema em tempo de execução, por meio da análise de traces distribuídos, spans e métricas coletadas durante o processamento das requisições.

6.3.1 Cenário de Teste

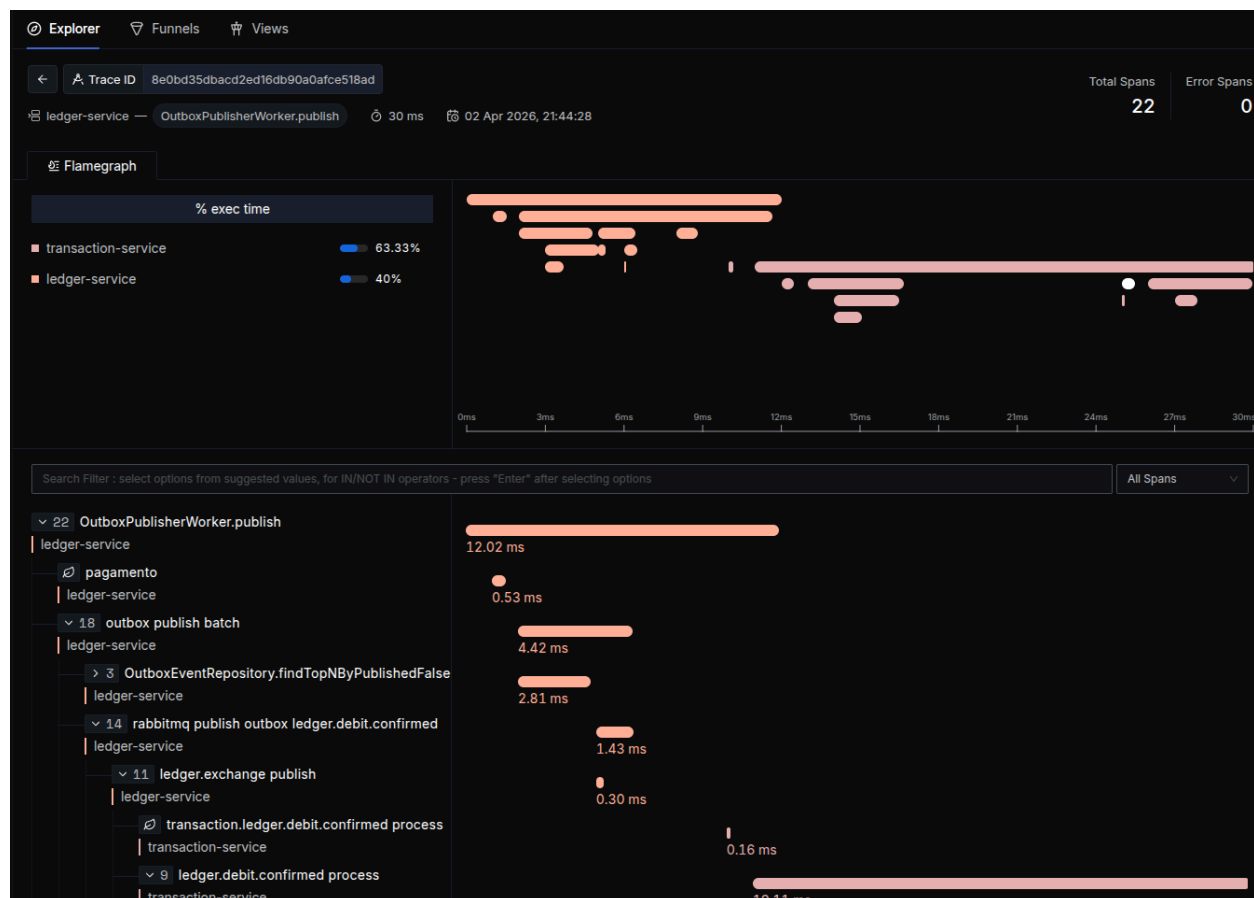
O teste consistiu na execução de uma transação no sistema, envolvendo os seguintes componentes:

- Transaction Service
- Ledger Service

- RabbitMQ (mensageria assíncrona)

6.3.2 Evidência de Execução

A figura abaixo apresenta o trace distribuído capturado no SigNoz durante a execução do fluxo de processamento de uma transação.



6.3.3 Análise do Trace

A análise do trace evidencia que:

- O fluxo foi iniciado no **Ledger Service**
- Houve comunicação assíncrona via **RabbitMQ**
- O evento foi processado pelo **Transaction Service**
- O componente **OutboxPublisherWorker**, de dentro do Ledger Service, foi responsável pela publicação dos eventos
- O trace apresentou **22 spans distribuídos**, indicando a execução completa do fluxo
- Não foram identificados erros (**0 error spans**)
- O tempo total de execução foi de aproximadamente **30 ms**

Além disso, é possível observar a divisão do tempo de processamento entre os serviços:

- Transaction Service: aproximadamente 63,33% do tempo de execução
- Ledger Service: aproximadamente 40% do tempo

Pode parecer estranho somar 63,33% + 40% resultar em 103,33%, porém esta soma não representa a realidade, pois parte do processo foi feito concomitantemente entre o Transaction Service e Ledger Service.

6.3.4 Critérios de Validação

O teste foi considerado bem-sucedido com base nos seguintes critérios:

- Existência de trace distribuído completo entre os serviços
- Presença de spans representando cada etapa do fluxo
- Ausência de erros durante a execução
- Comunicação assíncrona funcionando corretamente
- Tempo de resposta dentro de limites aceitáveis [Durou apenas 30ms, como visto na imagem, o que é excelente por se tratar de requisição entre containeres (conversa entre APIs) em menos de 100ms, porém este teste em relação ao tempo não é muito válido ao considerar que todos estão em localhost.]

6.3.5 Conclusão

Os resultados obtidos demonstram que a integração entre os serviços Transaction Service, Ledger Service e RabbitMQ está funcionando corretamente.

A utilização de observabilidade permitiu validar não apenas a comunicação entre os serviços, mas também a sequência de execução, o tempo de processamento e a ausência de falhas no fluxo distribuído.

Essa abordagem se mostrou eficaz para análise de sistemas distribuídos, fornecendo evidências concretas do comportamento da aplicação em ambiente real.

6.4 Conclusão dos Testes Não Funcionais

Os testes não funcionais executados mostraram que o sistema possui boa capacidade de resposta média, mas ainda apresenta fragilidades importantes sob carga e em cenários de falha induzida.

No teste de carga, o principal problema identificado foi a taxa elevada de falhas HTTP. No teste de resiliência, embora o sistema tenha mantido parte das funcionalidades operacionais, houve falhas relevantes em um endpoint. No de observável integração, apresenta integração sem erros entre componentes do sistema em 30ms, o que é excelente (Porém este teste em relação ao tempo não é muito válido ao considerar que todos estão em localhost.)

Assim, os resultados obtidos são valiosos para orientar melhorias na robustez da aplicação, na estabilidade dos fluxos internos e no tratamento de falhas entre serviços.